Manual de Instalación, Configuración y Despliegue

Backend SOAP — API Infracciones Vehiculares

Este manual proporciona una guía detallada paso a paso para configurar, compilar y ejecutar desde cero el servicio backend SOAP de API Infracciones Vehiculares.

1. Requisitos del Sistema y Requisitos Previos

Antes de comenzar, asegúrate de contar con los siguientes componentes en tu entorno de desarrollo:

- Java Development Kit (JDK): Versión 21 (se recomienda Temurin o Amazon Corretto).
- Base de Datos: MySQL Server 8.x (local, contenedor Docker o servidor remoto).
- Herramienta de Base de Datos: MySQL Workbench, DBeaver o cliente CLI de MySQL.
- Control de versiones: Git.
- Cliente de Pruebas API: Postman o SoapUI.
- Gestor de Construcción: Maven (incluido automáticamente dentro del proyecto mediante Maven Wrapper mvnw).



Verificación de la versión de Java

Abre tu terminal (PowerShell o CMD) y ejecuta:

java -version

> Nota: La salida debe confirmar la versión 21.x.x.

Si deseas trabajar de forma colaborativa con tu equipo de desarrollo, puedes crear o vincular un repositorio remoto en GitHub siguiendo estos pasos:

1. Crear un nuevo repositorio en GitHub:

    - Ve a GitHub y crea un nuevo repositorio (puedes dejarlo privado o público). No inicialices el repositorio con un README, `.gitignore` ni licencia, para evitar conflictos de historial.
2. Vincular y subir tu código local: Abre PowerShell o tu terminal en la raíz del proyecto y ejecuta:
3. PowerShell

```
# Si aún no has iniciado Git en el proyecto local
git init

# Agregar el origen remoto (reemplaza con la URL de tu nuevo repositorio)
git remote add origin https://github.com/TU_USUARIO/TU_REPOSITORIO.git

# Renombrar la rama principal a main
git branch -M main

# Guardar y subir los cambios iniciales
git add .
git commit -m "Initial commit - Backend SOAP Infracciones Vehiculares"
git push -u origin main
```

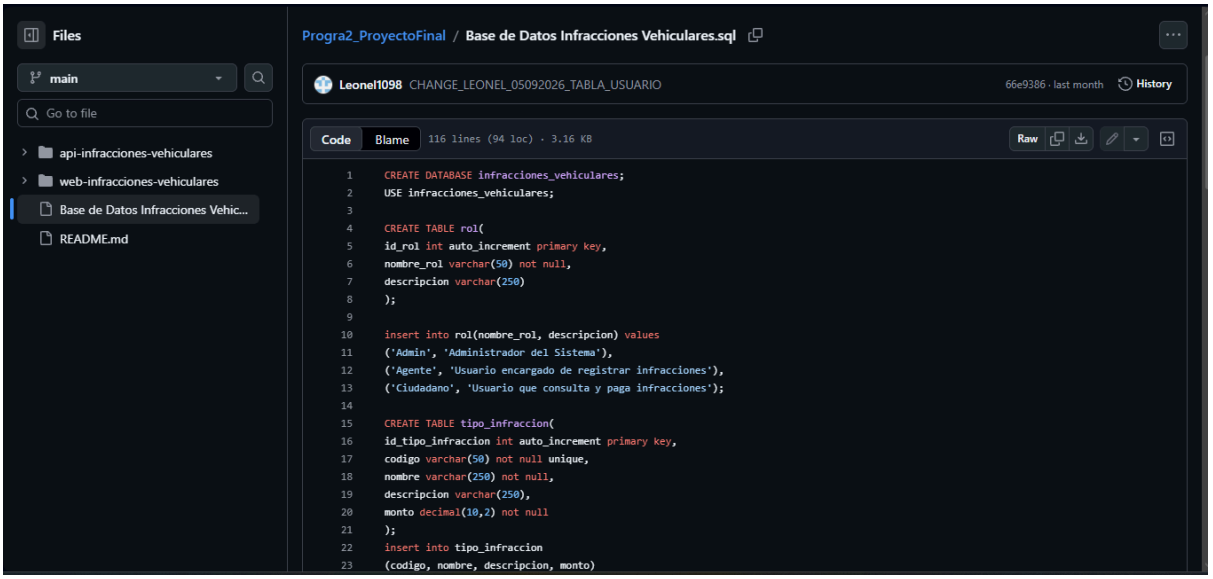
4. Buenas prácticas para el equipo:
    - Credenciales seguras: Asegúrate de que el archivo `src/main/resources/application.properties` se mantenga en el `.gitignore` y nunca se suba al repositorio remoto con contraseñas reales. Cada integrante del equipo debe crear su propio `application.properties` local basándose en `application.properties.example`.
    - Ramas por característica (*Feature Branches*): Para evitar conflictos en la rama `main`, se recomienda que cada desarrollador trabaje en su propia rama antes de integrar los cambios mediante *Pull Requests*:
    - PowerShell

```
# Crear y cambiar a una nueva rama
git checkout -b feature/nueva-funcionalidad

# Subir la rama a GitHub
```

    - git push origin feature/nueva-funcionalidad

Ejemplo:



Paso 1: Clonar el Repositorio

Abre la terminal en la carpeta donde deseas ubicar el proyecto y ejecuta los siguientes comandos:

git clone https://github.com/Leonel1098/Progra2_ProyectoFinal.git
cd Progra2_ProyectoFinal\api-infracciones-vehiculares

Paso 2: Configuración de la Base de Datos MySQL

1. Abre MySQL Workbench o tu cliente de base de datos preferido e inicia sesión.
2. Copia y ejecuta el siguiente script DDL/DML completo para inicializar el esquema infracciones_vehiculares:

```
CREATE DATABASE IF NOT EXISTS infracciones_vehiculares;
USE infracciones_vehiculares;

CREATE TABLE rol(
    id_rol INT AUTO_INCREMENT PRIMARY KEY,
    nombre_rol VARCHAR(50) NOT NULL,
    descripcion VARCHAR(250)
);

INSERT INTO rol(nombre_rol, descripcion) VALUES
('Admin', 'Administrador del Sistema'),
('Agente', 'Usuario encargado de registrar infracciones'),
('Ciudadano', 'Usuario que consulta y paga infracciones');
```

```
CREATE TABLE tipo_infraccion(
   id_tipo_infraccion INT AUTO_INCREMENT PRIMARY KEY,
   codigo VARCHAR(50) NOT NULL UNIQUE,
   nombre VARCHAR(250) NOT NULL,
   descripcion VARCHAR(250),
   monto DECIMAL(10,2) NOT NULL
);

INSERT INTO tipo_infraccion (codigo, nombre, descripcion, monto) VALUES
('INF-001', 'Exceso de velocidad', 'Conducir por encima del límite permitido', 500.00),
('INF-002', 'Estacionamiento prohibido', 'Estacionar en un área donde está prohibido',
250.00),
('INF-003', 'No portar licencia', 'Conducir sin portar la licencia correspondiente', 300.00);

CREATE TABLE estado_infraccion(
   id_estado INT AUTO_INCREMENT PRIMARY KEY,
   nombre_estado VARCHAR(250) NOT NULL
);

INSERT INTO estado_infraccion (nombre_estado) VALUES
('Pendiente'),
('Pagada'),
('Vencida'),
('Anulada');

CREATE TABLE regla_descuento(
   id_regla INT AUTO_INCREMENT PRIMARY KEY,
   porcentaje DECIMAL(10,2) NOT NULL,
   dias_limite INT NOT NULL,
   estado BOOLEAN NOT NULL
);

INSERT INTO regla_descuento (porcentaje, dias_limite, estado) VALUES (50.00, 4, TRUE);

CREATE TABLE persona(
   id_persona INT AUTO_INCREMENT PRIMARY KEY,
   dpi_persona VARCHAR(20) NOT NULL UNIQUE,
   nombre_persona VARCHAR(250) NOT NULL,
   apellido_persona VARCHAR(250) NOT NULL,
   telefono VARCHAR(50) NOT NULL,
   direccion VARCHAR(250) NOT NULL
);

CREATE TABLE usuario(
   id_usuario INT AUTO_INCREMENT PRIMARY KEY,
   usuario VARCHAR(20) NOT NULL UNIQUE,
   contraseña VARCHAR(250) NOT NULL,
```

```
    estado BOOLEAN NOT NULL,
    id_persona INT NOT NULL UNIQUE,
    id_rol INT NOT NULL,
    FOREIGN KEY (id_persona) REFERENCES persona(id_persona),
    FOREIGN KEY (id_rol) REFERENCES rol(id_rol)
);

CREATE TABLE vehiculo(
    id_vehiculo INT AUTO_INCREMENT PRIMARY KEY,
    placa_vehiculo VARCHAR(50) NOT NULL UNIQUE,
    marca VARCHAR(50) NOT NULL,
    modelo VARCHAR(50) NOT NULL,
    color VARCHAR(50) NOT NULL
);

CREATE TABLE infraccion(
    id_infraccion INT AUTO_INCREMENT PRIMARY KEY,
    numero_infraccion VARCHAR(50) NOT NULL UNIQUE,
    fecha_infraccion DATE NOT NULL,
    direccion_infraccion VARCHAR(250) NOT NULL,
    monto_original DECIMAL(10,2) NOT NULL,
    id_persona INT NOT NULL,
    id_vehiculo INT NOT NULL,
    id_usuario INT NOT NULL,
    id_tipo_infraccion INT NOT NULL,
    id_estado INT NOT NULL,
    FOREIGN KEY(id_persona) REFERENCES persona(id_persona),
    FOREIGN KEY(id_vehiculo) REFERENCES vehiculo(id_vehiculo),
    FOREIGN KEY(id_usuario) REFERENCES usuario(id_usuario),
    FOREIGN KEY(id_tipo_infraccion) REFERENCES tipo_infraccion(id_tipo_infraccion),
    FOREIGN KEY(id_estado) REFERENCES estado_infraccion(id_estado)
);

CREATE TABLE pago(
    id_pago INT AUTO_INCREMENT PRIMARY KEY,
    id_infraccion INT NOT NULL,
    monto_inicial DECIMAL(10,2) NOT NULL,
    porcentaje_descuento DECIMAL(5,2) NOT NULL,
    monto_descuento DECIMAL(10,2) NOT NULL,
    monto_pagado DECIMAL(10,2) NOT NULL,
    fecha_pago DATE NOT NULL,
    FOREIGN KEY(id_infraccion) REFERENCES infraccion(id_infraccion)
);
```

3. Verifica que la creación de las 9 tablas haya sido exitosa:

```
SHOW TABLES;
```



Paso 3: Configuración del Archivo de Propiedades

El backend utiliza una plantilla de configuración para proteger tus credenciales locales.

1. Duplica la plantilla de ejemplo para crear el archivo activo:

En Windows (PowerShell):
Copy-Item src\main\resources\application.properties.example
src\main\resources\application.properties

-

En Linux / macOS:
cp src/main/resources/application.properties.example
src/main/resources/application.properties

-
2. Abre `src/main/resources/application.properties` en tu editor de texto y edita los valores de conexión:

spring.application.name=api-infracciones-vehiculares

spring.datasource.url=jdbc:mysql://localhost:3306/infracciones_vehiculares?useSSL=false&
serverTimezone=America/Guatemala&allowPublicKeyRetrieval=true
spring.datasource.username=TU_USUARIO_MYSQL
spring.datasource.password=TU_CONTRASENA_MYSQL
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.sql.init.mode=never

cxf.servlet.enabled=true
cxf.servlet.mapping=/services/*

Paso 4: Compilación y Verificación del Proyecto

Ejecuta el ciclo de construcción usando Maven Wrapper para compilar las fuentes y verificar que no existan errores:

En Windows:
.\mvnw.cmd clean test

En Linux / macOS:
./mvnw clean test

Resultado esperado:

[INFO] BUILD SUCCESS
[INFO] --
[INFO] Total time: X.XXX s

Paso 5: Ejecución del Servicio Backend

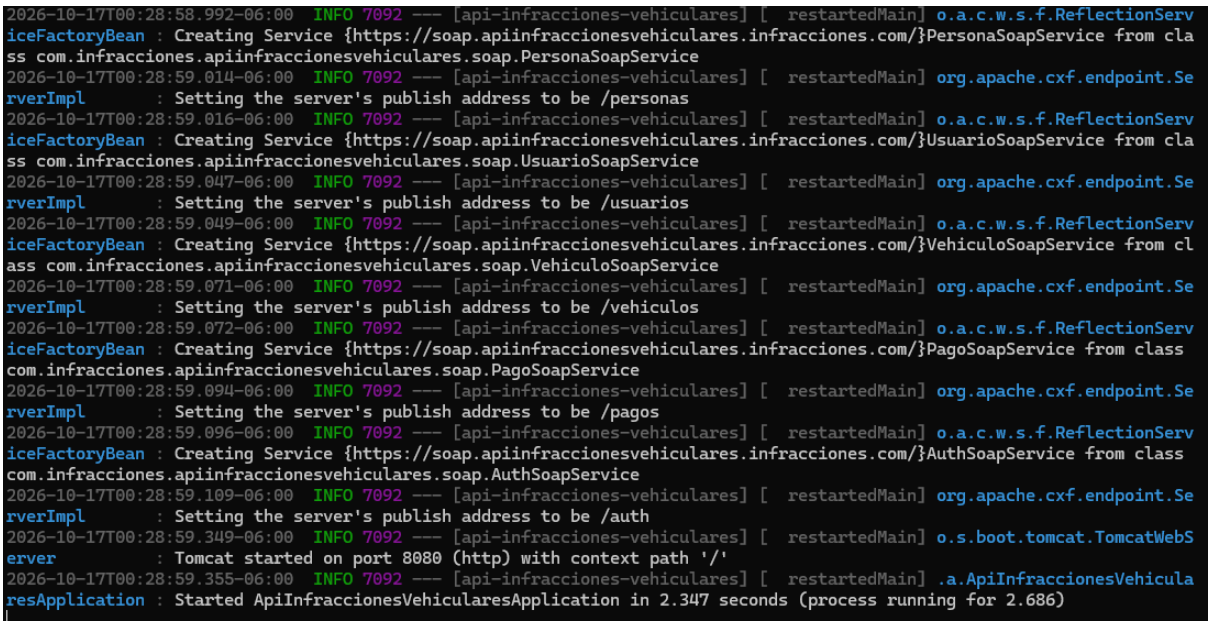
Una vez compilado correctamente, inicia la aplicación Spring Boot:

En Windows:
.\mvnw.cmd spring-boot:run

En Linux / macOS:
./mvnw spring-boot:run

Cuando el proceso termine de cargar, verás un mensaje confirmando que el servidor está escuchando en el puerto local 8080:

Started ApiInfraccionesVehicularesApplication in X.XXX seconds (process running for X.XXX)

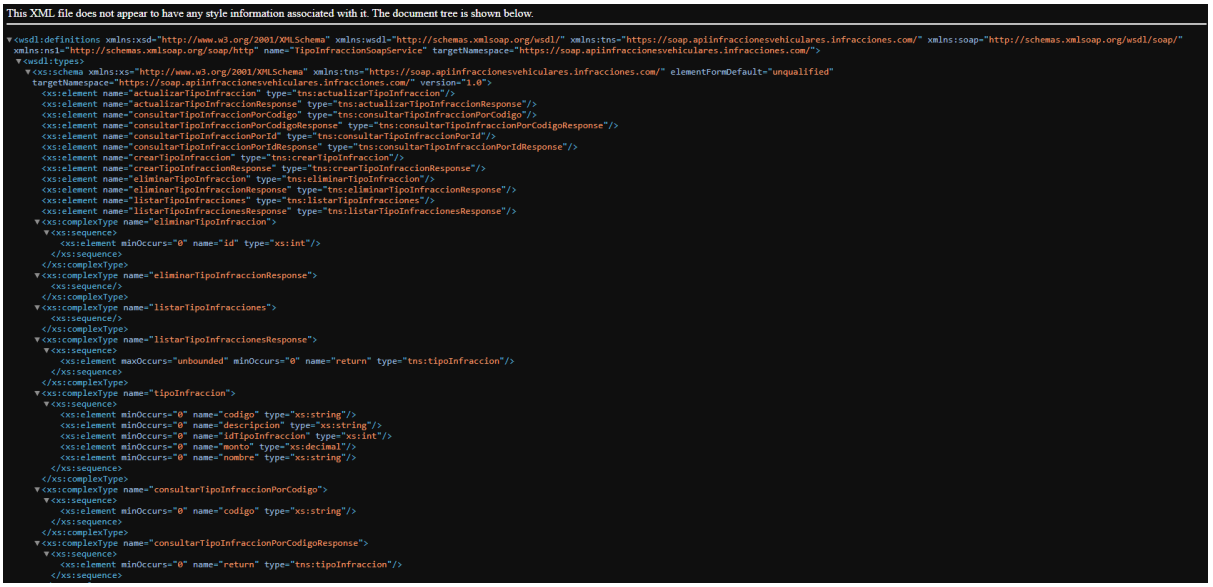


Paso 6: Verificación de Servicios SOAP y Endpoints

Para confirmar que Apache CXF ha publicado los servicios correctamente, abre tu navegador e ingresa a las siguientes direcciones WSDL:

Servicio	URL del Contrato WSDL
Tipos de Infracción	`http://localhost:8080/services/tipoinfracciones?wsdl`
Personas	`http://localhost:8080/services/personas?wsdl`
Usuarios	`http://localhost:8080/services/usuarios?wsdl`
Vehículos	`http://localhost:8080/services/vehiculos?wsdl`

Infracciones	`http://localhost:8080/services/infracciones?wsdl`
Pagos	`http://localhost:8080/services/pagos?wsdl`



Paso 7: Prueba de Operaciones (Postman)

Opción A: Importar la colección preconfigurada

1. Abre Postman.
2. Selecciona File → Import.
3. Elige el archivo ubicado en la carpeta del repositorio: `postman/Proyecto Infracciones.postman_collection.json`.

Opción B: Crear una petición manual

Para probar la ejecución de una operación directamente:

1. Método: `POST`
2. URL: `http://localhost:8080/services/tipoinfracciones`
3. Headers:
    - `Content-Type`: `text/xml; charset=UTF-8`
4. Body (`raw` -> `XML`):

```
<soapenv:Envelope
   xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
   xmlns:tns="https://soap.apiinfraccionesvehiculares.infracciones.com/">
```

```
   <soapenv:Header/>
   <soapenv:Body>
      <tns:listarTipoInfracciones/>
   </soapenv:Body>
</soapenv:Envelope>
```

Resolución de Problemas Frecuentes

Falla / Error	Causa Probable	Solución
`Failed to determine a suitable driver class`	No se creó o no se configuró el archivo `application.properties`.	Sigue el Paso 3 para copiar y completar el archivo de propiedades.
`Communications link failure`	El servicio de MySQL está apagado o el puerto `3306` está bloqueado.	Inicia el servicio de MySQL y valida la conectividad en la URL JDBC.
`Access denied for user '...'@'localhost'`	Las credenciales de MySQL en `application.properties` son incorrectas.	Revisa y corrige el usuario y contraseña del motor MySQL.
El WSDL carga sin operaciones	Mala configuración del Bean en `SoapConfig.java`.	Asegúrate de publicar la interfaz del paquete `soap` y no la implementación del servicio.

Manual de Configuración y Despliegue: Frontend Web de Infracciones (Spring Boot / JSP)

1. Descripción General del Sistema

La aplicación web de Infracciones Vehiculares opera bajo una arquitectura MVC (Modelo-Vista-Controlador) construida con Spring Boot y JSP. Funciona como una capa de interfaz de cliente liviana que no posee base de datos propia ni lógica de persistencia; todas las operaciones de negocio y almacenamiento son delegadas a un API backend SOAP centralizado.

[Navegador Web] ──HTTP──> [Frontend Spring Boot :8081] ──SOAP/HTTP──> [API Backend :8080] ──> [Base de Datos MySQL]

2. Requisitos Previos del Entorno

Antes de iniciar la configuración, asegúrese de cumplir con los siguientes componentes en el equipo de desarrollo o servidor de despliegue:

- Java Development Kit (JDK): Versión 17 o superior.
- API Backend en ejecución: El servicio SOAP principal (api-infracciones-vehiculares) debe estar activo y accesible en el puerto 8080.
- Herramientas de red: Conectividad local para validar los contratos WSDL de los servicios (ej. http://localhost:8080/services/infracciones?wsdl).

3. Configuración de Propiedades de Conexión

La aplicación se comunica con el backend mediante endpoints SOAP configurados en las propiedades del sistema.

1. Localice la plantilla de configuración predeterminada ubicada en la ruta:

   src/main/resources/application.properties.example

2. Genere una copia de este archivo nombrándola application.properties en el mismo directorio (este archivo está excluido del control de versiones por seguridad).
3. Verifique y ajuste los parámetros de conexión según el entorno de despliegue:

server.port=8081

soap.infracciones.url=http://localhost:8080/services/infracciones

soap.tipos.url=http://localhost:8080/services/tipoinfracciones

soap.personas.url=http://localhost:8080/services/personas

soap.vehiculos.url=http://localhost:8080/services/vehiculos

soap.auth.url=http://localhost:8080/services/auth

soap.pagos.url=http://localhost:8080/services/pagos

soap.usuarios.url=http://localhost:8080/services/usuarios

4. Compilación y Ejecución del Proyecto

El proyecto incluye Maven Wrapper (mvnw), lo que evita la necesidad de instalar Maven manualmente en el sistema operativo.

Opción A: Compilación y ejecución directa (Entorno de desarrollo)

Abra una terminal (PowerShell o consola equivalente) en la raíz del proyecto y ejecute:

.\mvnw.cmd spring-boot:run

Opción B: Compilación omitiendo pruebas unitarias (Generación de paquetes)

Si requiere compilar el código fuente sin ejecutar las pruebas automatizadas, utilice:

.\mvnw.cmd -DskipTests compile

Una vez iniciado el servicio, abra su navegador web e ingrese a la ruta local configurada:

URL de acceso: `http://localhost:8081/`

5. Arquitectura de Seguridad y Control de Acceso

- Tokens de Sesión: Las solicitudes autenticadas enviadas al API incorporan el token de sesión activo mediante la cabecera HTTP estándar Authorization: Bearer <token>.
- Protección CSRF: Las rutas HTTP de tipo POST asociadas a formularios protegidos implementan validación de tokens contra ataques de falsificación de peticiones en sitios cruzados.
- Validación de Roles**:** La seguridad de la interfaz oculta elementos visuales según el rol (Ciudadano, Agente, Admin); no obstante, la autorización y rechazo estricto de privilegios se ejecuta de forma centralizada en el API Backend.